



Informatics

A Comprehensive Synthesis of Modelling Approaches for Smart Contracts on Blockchain: Bridging Economic Transactions and Business Informatics

BACHELOR'S THESIS

submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science

in

Business Informatics

by

Timo Maksan

Registration Number 12124558

to the Faculty of Informatics

at the TU Wien

Advisor: Univ.Prof. / PhD Henderik Proper

Vienna, June 22, 2024

Timo Maksan

Henderik Proper

Erklärung zur Verfassung der Arbeit

Timo Maksan

Hiermit erkläre ich, dass ich diese Arbeit selbständig verfasst habe, dass ich die verwendeten Quellen und Hilfsmittel vollständig angegeben habe und dass ich die Stellen der Arbeit – einschließlich Tabellen, Karten und Abbildungen –, die anderen Werken oder dem Internet im Wortlaut oder dem Sinn nach entnommen sind, auf jeden Fall unter Angabe der Quelle als Entlehnung kenntlich gemacht habe.

Wien, 22. Juni 2024

Timo Maksan

Inhaltsverzeichnis

Inhaltsverzeichnis	iii
1 Introduction	1
1.1 Abstract	1
1.2 Foreword	1
1.3 Research questions	2
1.4 Method	2
1.5 Structure	2
2 Background	4
2.1 Blockchain	4
2.2 Smart Contracts	4
2.3 Integration of the Two	5
3 Method	6
3.1 Selection of Papers	6
3.2 Analytical Framework	6
4 Results	9
4.1 "Towards Automatic Smart Contract Generation from DEMO Models: A Case in Logistics using Hyperledger" Paper	9
4.2 "Design and development of Smart Contracts for E-government through Value and Business Process Modelling" Paper	14
5 Evaluation and Discussion	20
5.1 Evaluation of the Selected Frameworks	20
6 Conclusion	25
Bibliography	26

Introduction

1.1 Abstract

This thesis presents an in-depth analysis of two methodologies for generating smart contracts: "Towards Automatic Smart Contract Generation from DEMO Models: A Case in Logistics Using Hyperledger" by Abreu, Aveiro, and Freitas, and "Design and Development of Smart Contracts for E-Government through Value and Business Process Modelling" by Gómez, Pérez-Blanco, Vara, De Castro, and Marcos. Both approaches aim to enhance the efficiency, transparency, and security of blockchain applications in logistics and e-government, respectively. The analysis is conducted using a structured framework evaluating each methodology's framework overview, technical implementation, mapping and translation processes, usability, validation, and testing. The thesis discusses the strengths and limitations of each approach, providing in-sights into their practical applications and potential for future improvement. The findings highlight the importance of formal models and domain-specific languages in streamlining smart contract development and emphasize the need for further research to address scalability, automation, and accessibility challenges.

1.2 Foreword

The advent of blockchain technology has revolutionized numerous sectors by introducing decentralized, secure, and transparent mechanisms for conducting transactions. Among its most promising applications is the implementation of smart contracts—self-executing contracts with the terms of the agreement directly written into code. These digital contracts facilitate, verify, and enforce the negotiation or performance of a contract, minimizing the need for intermediaries and reducing transaction costs [1]. However, the complexity of smart contracts and their underlying blockchain infrastructure poses significant challenges in terms of modelling and understandability, particularly for stakeholders

who lack technical expertise. Numerous modelling frameworks have been proposed to address these challenges, each offering different perspectives and methodologies for translating economic transactions into smart contracts [2, 1]. This thesis aims to analyse and synthesize existing modelling frameworks for smart contracts, focusing on their application in economic transactions and business informatics. By drawing on the work of re-searchers and examining various theoretical perspectives, this study seeks to construct a unified understanding that enhances the clarity and usability of smart contracts. Additionally, the research will explore the representation of these contracts to ensure they are understandable to stakeholders, thereby bridging the gap between technical complexity and practical application.

1.3 Research questions

1. How do existing modelling frameworks, such as DEMO and business process modelling, facilitate the automatic generation of smart contracts?
2. To what extent do these frameworks ensure that non-technical stakeholders can comprehend and validate the smart contracts generated?
3. How can existing frameworks for smart contract generation be improved to enhance automation, validation, and stakeholder engagement?

1.4 Method

The structure of this thesis is as follows: First, an overview of the blockchain technology and smart contract structure, followed by a detailed review of the existing literature on smart contract modelling frameworks, will be presented. This will be followed by an analysis of the strengths and weaknesses of different approaches, culminating in the synthesis of a cohesive framework. Finally, the study will address the understandability of smart contracts, proposing methods to improve their representation for non-technical stakeholders. Through this comprehensive analysis, the thesis aims to contribute to the ongoing discourse on blockchain technology and smart contracts, providing valuable insights for both academia and industry.

1.5 Structure

The paper will be split in the following chapters/subchapters:

1. Introduction
2. Background
 - Blockchain

- Smart Contracts
- Integration of the Two

3. Method

4. Results

5. Evaluation and Discussion

6. Conclusion

7. References

Background

2.1 Blockchain

Blockchain technology is a decentralized digital ledger that records transactions across multiple computers, ensuring the security and transparency of data. Each block in the blockchain contains a list of transactions and is linked to the previous block through a cryptographic hash, forming a chain. This structure makes it nearly impossible to alter any information without affecting all subsequent blocks, thus maintaining data integrity and trustworthiness. Unlike traditional centralized databases, blockchain operates on a peer-to-peer network where each participant maintains a copy of the entire ledger. This setup enhances accountability and security, as the use of cryptographic algorithms ensures data is secure and tamper-proof. Additionally, blockchain's transparency allows all transactions to be visible to all participants in the network, providing a permanent and unchangeable record of transactions [3, 4].

2.2 Smart Contracts

Smart contracts are self-executing contracts with the terms of the agreement directly written into code. These contracts automatically execute and enforce the terms when predefined conditions are met, eliminating the need for intermediaries. The precision of code ensures that contract terms are executed exactly as written, while the underlying blockchain technology's security features make smart contracts resistant to tampering and fraud. Operating on platforms like Ethereum, smart contracts benefit from the cryptographic security and transparency of blockchain, providing a secure and trustworthy mechanism for executing agreements. The transparency inherent in blockchain allows all parties to see the contract terms and verify their execution, reducing the likelihood of dispute [2, 1].

2.3 Integration of the Two

The integration of smart contracts with blockchain technology creates a powerful tool for automating and securing transactions in a decentralized manner. This combination enables complex applications, such as decentralized finance (DeFi), supply chain management, and digital identity verification, to be implemented more efficiently and securely. However, the complexity of smart contracts and the technical intricacies of blockchain pose significant challenges in terms of modelling, representation, and understandability for non-technical stakeholders. To address these issues, various modelling frameworks have been developed to translate economic transactions into smart contracts that are both executable and comprehensible to all parties involved. This thesis will explore these modelling frameworks, analysing their effectiveness and proposing enhancements to improve the usability and clarity of smart contracts in practical applications [3, 4, 2, 1].

Method

This section outlines the methodological approach for analyzing the two selected papers:

- *Towards Automatic Smart Contract Generation from DEMO Models: A Case in Logistics using Hyperledger* by Abreu, Aveiro, and Freitas
- *Design and Development of Smart Contracts for E-Government through Value and Business Process Modelling* by Gómez, Pérez-Blanco, Vara, De Castro, and Marcos

The analysis will focus on evaluating the frameworks and methodologies proposed in each paper, using specific criteria to assess their effectiveness, applicability, and potential for improvement [5, 6, 7].

3.1 Selection of Papers

The two papers were selected based on their relevance to the topic of smart contract modelling and their use of distinct methodologies and frameworks. Both papers address the integration of smart contracts into different domains—logistics and e-government—providing a diverse perspective on the practical applications and challenges associated with smart contract generation. Additionally, these papers were recommended by my mentor, who guided the selection process. During my own search for further relevant papers, I found that other available studies either did not focus specifically on the topic of smart contract modelling or were too broad in scope to provide the focused insights needed for this analysis.[8].

3.2 Analytical Framework

The analytical framework was developed based on the guidelines provided in the source Research Paper Analysis: How to Analyze a Research Article + Example by HelpfulPapers

[8]. This source offered a comprehensive approach to analyzing research articles, which was adapted to evaluate the specific aspects of the frameworks and methodologies discussed in the selected papers. The framework includes the following criteria:

3.2.1 Framework Overview and Objectives

- **Description:** A detailed description of each framework's core components, methodologies, and objectives.
- **Objectives:** The specific goals the framework aims to achieve within its domain (e.g., logistics, e-government).

3.2.2 Technical Implementation

- **Methodology:** The approach and steps taken to implement the smart contract generation process.
- **Technological Tools:** The technologies and platforms used (e.g., Hyperledger Fabric, DEMO, business process modelling tools).

3.2.3 Mapping and Translation Process

- **Mapping Techniques:** How business models are mapped to smart contract code.
- **Translation Accuracy:** The accuracy and efficiency of translating business rules into executable smart contracts.

3.2.4 Usability and Understandability

- **Stakeholder Comprehension:** How well the generated smart contracts can be understood by non-technical stakeholders.
- **Documentation and Explanation:** The clarity and comprehensiveness of the documentation provided for stakeholders.

3.2.5 Validation and Testing

- **Validation Methods:** The methods used to validate the correctness and functionality of the smart contracts.
- **Testing Scenarios:** The scenarios and case studies used to test the smart contracts in practical applications.

3.2.6 Advantages and Limitations

- **Strengths:** The strengths and advantages of each framework.
- **Weaknesses:** The limitations and potential areas for improvement [8].

Results

4.1 "Towards Automatic Smart Contract Generation from DEMO Models: A Case in Logistics using Hyperledger" Paper

The paper explores the integration of DEMO (Design & Engineering Methodology for Organizations) models with Hyperledger Fabric's smart contracts written in Go. The primary objective is to automate the generation of smart contracts using DEMO's Action Rule Specification (ARS), enhancing efficiency and reducing errors in blockchain-based applications. By translating DEMO models into executable chaincode, the research aims to facilitate complex business logic implementation, data validation, and transaction management on blockchain platforms like Hyperledger Fabric. This approach combines formal enterprise modelling with blockchain technology to streamline contract development and improve organizational efficiency [5, 9].

The authors employ a structured methodology to integrate DEMO models with smart contracts written in Go for Hyperledger Fabric. Here's a breakdown of their methodology and the technical tools used:

4.1.1 Methodology

1. DEMO Modelling:

- Action Rule Specification (ARS): DEMO's ARS is used to formalize business rules and processes. It defines actions and conditions that govern organizational transactions.
- DEMO Transactions: Transactions in DEMO represent atomic operations within an organization, describing who initiates the action and what changes occur in the system [5, 9].

2. Smart Contract Generation:

- Mapping DEMO to Smart Contracts: The researchers map DEMO transaction patterns into executable smart contracts. This involves translating DEMO's ARS into Go programming language constructs suitable for Hyperledger Fabric's chaincode [5, 9].

3. Implementation on Hyperledger Fabric:

- Hyperledger Fabric: Chosen for its modular architecture and support for private, permissioned blockchains.
- Chaincode Development: Smart contracts are implemented as chaincode in Go, utilizing Fabric's SDKs and APIs for transaction processing, data management, and consensus [5, 9].

4. Testing and Validation:

- Validation of Generated Contracts: The generated smart contracts are tested to ensure they accurately reflect the business logic specified in DEMO models.
- Functional Testing: Validation includes functional tests to verify transaction integrity, data consistency, and adherence to predefined business rules [5, 9].

4.1.2 Technical Tools

• **DEMO (Design & Engineering Methodology for Organizations):**

- Used for formal modelling of organizational structures, processes, and transactions.
- Specifically, the Action Rule Specification (ARS) within DEMO is leveraged to define transactional behavior [5, 9].

• **Hyperledger Fabric:**

- An open-source blockchain framework hosted by the Linux Foundation.
- Utilizes a modular architecture with components for consensus, membership services, and smart contract execution (chaincode).
- Provides SDKs for Go (and other languages) to develop, deploy, and interact with smart contracts [5, 9].

• **Go Programming Language:**

- Chosen for its efficiency and performance in developing smart contracts for Hyperledger Fabric.
- Go's simplicity and concurrency features are leveraged to implement complex business logic as defined by DEMO's ARS.

4.1. "Towards Automatic Smart Contract Generation from DEMO Models: A Case in Logistics using Hyperledger" Paper

property	STRING NOTE: has to be an existent property specified in table property
validation_condition	[NOT] validation_condition_type [EXTRA_FIELD_1 [EXTRA_FIELD_2]] [user_output] NOTE: The not, extrafield1 and extrafield2 fields will appear depending on the validation_condition_type chosen. EXTRAFIELD1 and EXTRAFIELD2 examples are the min and max fields if the validation_condition_type is "Belongs Range".
validation_condition_type	REQUIRED IS_NUMBER IS_INTEGER EQUAL_TO MAX_WORD_LENGTH LESS_EQUAL HIGHER_EQUAL HIGHER_THAN LESS_THAN MIN_LENGTH BELONG_SRANGE MAX_LENGTH MIN_WORD_LENGTH HAS_CHARACTER REG_EXPRESSION HAS_WORD IS_EMAIL IS_URL CUSTOM_VALIDATION
compute_expression	term {compute_operator term}-
compute_operator	"+" "-" "*" "/" NOTE: for now simple mathematical operators, later maybe more...

Figure 4.1: Mapping of Actions to Code Example

The mapping process from business models, specifically DEMO models, to smart contract code involves a systematic approach to capture and translate organizational processes and transactions into executable blockchain logic. DEMO's Action Rule Specification (ARS) provides a structured framework for defining business rules and transactional behavior within organizational contexts. To map these models to smart contract code, the methodology focuses on identifying key transaction patterns and their corresponding actions as defined in DEMO's ARS [5, 9].

Each transactional pattern is meticulously mapped to specific functions and data structures within the smart contract written in Go for Hyperledger Fabric. This mapping ensures that the business logic encoded in DEMO models is accurately represented and executed within the blockchain environment. Techniques such as object-oriented design principles and modularization are employed to enhance code readability, maintainability, and scalability of the resulting smart contracts. By leveraging these mapping techniques, the methodology ensures that the blockchain-based applications adhere closely to the intended business processes and rules defined in DEMO [5, 9].

The translation of business rules from DEMO models into executable smart contracts is a critical aspect of ensuring the integrity and reliability of blockchain-based applications. Accuracy in translation involves faithfully capturing the semantics and conditions specified in DEMO's ARS and transforming them into executable code in Go. This process includes rigorous validation and verification steps to ensure that the smart contracts accurately enforce the defined business rules without ambiguity or deviation [5, 9].

Efficiency in translation is achieved through optimization techniques in code implementation, leveraging Go's concurrency features and Hyperledger Fabric's modular architecture



Figure 4.2: DEMO Action Rule: “When” block

for transaction processing and data management. The methodology emphasizes testing at each stage of translation to validate the functionality and performance of the smart contracts. Functional testing verifies that the translated smart contracts correctly handle transactional flows and adhere to predefined business rules, ensuring that the blockchain applications operate reliably in real-world scenarios [5, 9].

By focusing on both accuracy and efficiency in translating business rules into executable smart contracts, the methodology aims to deliver robust blockchain solutions that enhance transparency, security, and efficiency in organizational transactions. This approach ensures that the blockchain-based applications developed align closely with the organizational models and requirements defined by DEMO, thereby facilitating seamless integration of blockchain technology into enterprise environments [5, 9].

The generated smart contracts prioritize understandability for non-technical stakeholders through clear structuring and documentation. Each contract is designed with a user-friendly interface, ensuring that stakeholders can easily grasp the contractual obligations and operational logic embedded within. This approach facilitates transparent communication and alignment of business objectives across diverse stakeholder groups. Comprehensive documentation accompanies each smart contract, aiming to provide stakeholders with clear insights into its functionality and operational parameters. The documentation outlines the contract’s purpose, key features, and transactional workflows in a straightforward manner, supplemented by illustrative examples where applicable. By emphasizing clarity and completeness in documentation, stakeholders are empowered to engage effectively with the smart contracts, fostering trust and facilitating informed decision-making within blockchain-based applications [5, 9].

The approach to validating smart contracts ensures correctness and functionality through rigorous methods such as code reviews, formal verification, and integration testing. Code reviews involve peer scrutiny to identify logical errors and ensure adherence to best practices and security standards. Formal verification employs mathematical proofs

to validate contract behavior against specified requirements, enhancing confidence in reliability and performance. Integration testing validates interactions with external systems, ensuring seamless functionality in diverse operational environments [5, 9].

```
func (s *SmartContract) AuctionCreateMain(stub shim.ChaincodeStubInterface,
    auction models.Auction, parcels []string) pb.Response {
    pseudoInitiateTransactionEnsureAtomicity()
    //ADD ACTIONS HERE
    (...)
    return shim.Success(nil)
}
```

Figure 4.3: Pseudo Code of Creating Auction Main Method

Smart contracts undergo comprehensive testing across diverse scenarios and case studies to validate their robustness in practical applications. Testing encompasses varying transaction volumes, boundary conditions, and edge cases to simulate real-world usage scenarios. Case studies explore contract performance under different user behaviors and environmental factors, ensuring resilience and scalability. By leveraging practical testing methodologies, we ensure that smart contracts meet operational expectations and deliver consistent performance in dynamic blockchain ecosystems [5, 9].

The research demonstrates the feasibility and benefits of automatic smart contract (SC) generation for Hyperledger Fabric using DEMO Action Rule (AR) models and DLCP Dynamic Search. This approach significantly enhances efficiency, reducing the time and resources needed for creating complex contracts. It supports automated data validation, error handling, and interaction management within the blockchain. A detailed study of the "Creation of Auction" transaction and the "Query Open Auctions" method validated the approach by developing a smart contract in Go, showcasing its effectiveness. The research confirmed that automatic SC generation could handle various scenarios, including data validation, entity relationships, iterative processes, sophisticated queries, and robust error management. DEMO AR models streamline contract development and provide structured file organization, enhancing performance and clarity in blockchain queries [5, 9].

However, the expressiveness of the DEMO language may limit advanced contract logic, indicating a need for enhancements to address blockchain-specific challenges. Tailored adjustments might be required for specific sectors to meet unique requirements. Future work should focus on integrating advanced functionalities, such as sophisticated filtering mechanisms in queries to provide richer datasets. As complexity increases, maintaining system efficiency and robustness is crucial. Optimizing index creation for DLCP Dynamic Search queries is also important. An ongoing initiative is a low-code platform to transpile DEMO models directly into Go code, simplifying blockchain implementation and automating essential API code generation. While promising, continuous refinement and evaluation are necessary to keep pace with evolving blockchain demands [5, 9].

4.2 “Design and development of Smart Contracts for E-government through Value and Business Process Modelling” Paper

In the paper, the framework discussed primarily revolves around INNoVaServ, a modelling toolkit designed to integrate various DSLs (Domain-Specific Languages) and facilitate the development of smart contracts within the context of e-government services. INNoVaServ is built upon the Eclipse IDE and leverages plugins that implement visual DSLs, initially using Eclipse Modelling Framework (EMF) and later transitioning to Sirius due to evolving technical support. The core components include an integrated Eclipse environment with graphical editors for different DSLs, supported by a model processor layer that enables functionalities like model validation, fixing, and transformations (M2M) using Epsilon languages (ETL, EOL). These components collectively aim to streamline service design processes and bridge the digital gap between different government professionals [6, 7].

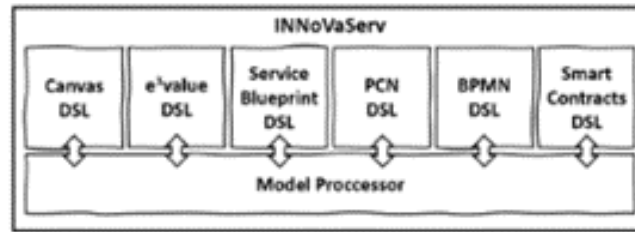


Figure 4.4: Simplified overview of INNoVaServ’s conceptual architecture

Objectives of the INNoVaServ framework are multifaceted. Firstly, it aims to enhance the efficiency and transparency of e-government services by integrating model-based tools that facilitate the analysis of requirements and implementation of digital services. Secondly, INNoVaServ strives to enable seamless integration of smart contracts into existing e-government frameworks, thereby reducing administrative overhead, enhancing security, and ensuring compliance with digital governance standards. Thirdly, the framework aims to support interoperability with other EMF/GMF-based tools, allowing for the combination of UML models and business management tasks within a unified modelling environment. Overall, INNoVaServ’s objectives are centered on enhancing the agility, transparency, and effectiveness of e-government service delivery through advanced modelling and smart contract integration capabilities [6, 7].

In the paper, the technical implementation of smart contract generation within the INNoVaServ framework is detailed through a systematic methodology and the utilization of specific technological tools. The approach begins with the integration of a DSL (Domain-Specific Language) named SmaC, developed using Xtext, a tool that facilitates the creation of textual DSLs. SmaC serves as the primary means to code smart contracts in Solidity, a programming language commonly used for Ethereum blockchain [6, 7].

The methodology employed involves defining a structured pattern within SmaC to enhance the readability and understanding of smart contract code. This structured

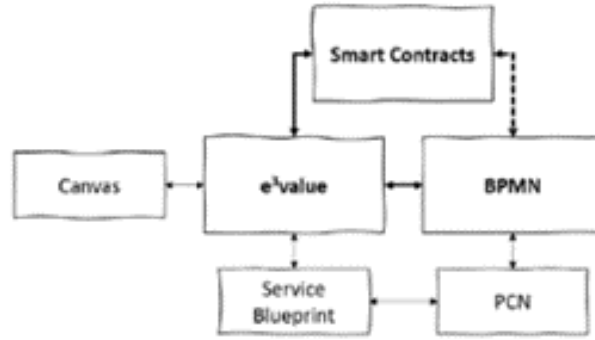


Figure 4.5: Generating smart contracts from high-level models

approach includes provisions for gas control during loop executions to prevent security vulnerabilities, as well as the incorporation of ad-hoc facilities such as code autocompletion, validation, and quick fixes to expedite the development process [6, 7].

Technologically, INNoVaServ leverages Eclipse-based tools including Eclipse Modeling Framework (EMF), Sirius, and Acceleo for model-driven development and transformation tasks. The use of Epsilon languages (ETL, EOL) facilitates the automated generation of smart contract skeletons from corresponding e3value models, which capture value exchanges and dependencies within e-government service processes [6, 7].

Moreover, the integration of SmaC with EMF enables the injection of smart contract models into EMF structures, allowing them to undergo model-based processing tasks. This approach not only ensures compatibility with existing EMF-based environments but also supports seamless interactions between different DSLs within the toolkit. Overall, the technical implementation within INNoVaServ combines robust DSL development methodologies with sophisticated model-driven techniques to streamline the integration of smart contracts into e-government service design and execution [6, 7].

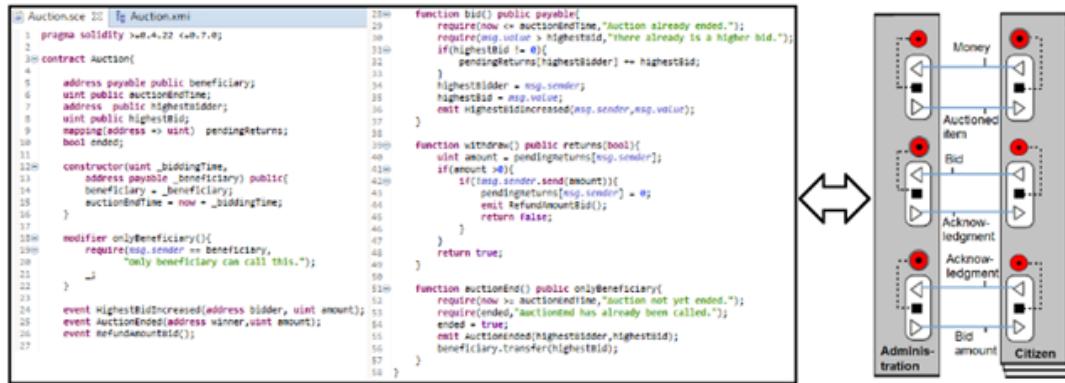


Figure 4.6: Correspondences between an auction smart contract and the correspondent e3 value model

The mapping techniques employed leverage model-driven development principles within the INNoVaServ framework. Business models, represented in various notations such as BPMN (Business Process Model and Notation) and e3value models, serve as foundational inputs. These models capture detailed business processes, value exchanges, and dependencies inherent in e-government service scenarios. Through model transformations facilitated by Epsilon languages (ETL, EOL), these business models are mapped to corresponding smart contract structures in the SmaC DSL. ETL supports many-to-many model transformations, enabling the conversion of abstract business logic into concrete smart contract functionalities. This mapping process ensures that the essential aspects of business rules, transactional flows, and value propositions are accurately reflected in the resulting smart contract code [6, 7].

The translation accuracy from business models to executable smart contracts is a critical aspect of the framework’s implementation. By utilizing DSL-based approaches like SmaC, which provides a structured pattern for coding smart contracts, the framework ensures that business rules are faithfully translated into Solidity code. The approach includes provisions for handling complexities such as gas control to prevent security vulnerabilities, ensuring that the translated smart contracts adhere to both functional requirements and security standards. The integration of model validation and automated fixing mechanisms further enhances translation accuracy by identifying and rectifying potential discrepancies between the business model specifications and their corresponding smart contract implementations. This systematic approach not only guarantees the fidelity of translation but also enhances the efficiency of the development process by reducing manual effort and potential errors [6, 7].

The framework places significant emphasis on ensuring that the generated smart contracts are comprehensible to non-technical stakeholders, such as government administrators and service managers. This is achieved through several strategic approaches. Firstly, the SmaC DSL, developed using Xtext, facilitates the creation of textual representations of smart contracts. These contracts are designed to follow a structured pattern that enhances readability and understandability. By adhering to predefined syntax and semantics, SmaC enables stakeholders to grasp the fundamental logic and operational flow of smart contracts without delving into the intricacies of Solidity code. Moreover, the use of high-level modeling notations like BPMN and e3value models provides visual representations that aid in conveying the business processes and value exchanges that underpin the smart contract logic. These visual models serve as intuitive interfaces for stakeholders to interact with and validate the operational behavior of smart contracts, fostering a clearer understanding of how blockchain technology supports e-government services [6, 7].

To complement the stakeholder comprehension efforts, the framework employs comprehensive documentation practices. The documentation accompanying the smart contracts is meticulously crafted to provide clarity on their purpose, functionality, and interaction within the broader e-government service ecosystem. This documentation spans from high-level overviews to detailed technical specifications, catering to different levels of

stakeholder expertise. For instance, executive summaries and conceptual diagrams are used to convey the overarching goals and benefits of implementing smart contracts in e-government services. At the same time, detailed explanations of smart contract functionalities, supported by annotated code snippets and flowcharts derived from BPMN and e3value models, offer deeper insights into the operational nuances and transactional flows encapsulated in the smart contracts. This multi-layered approach ensures that stakeholders, regardless of their technical proficiency, can access the information they need to understand how smart contracts enhance transparency, efficiency, and security in e-government service delivery [6, 7].

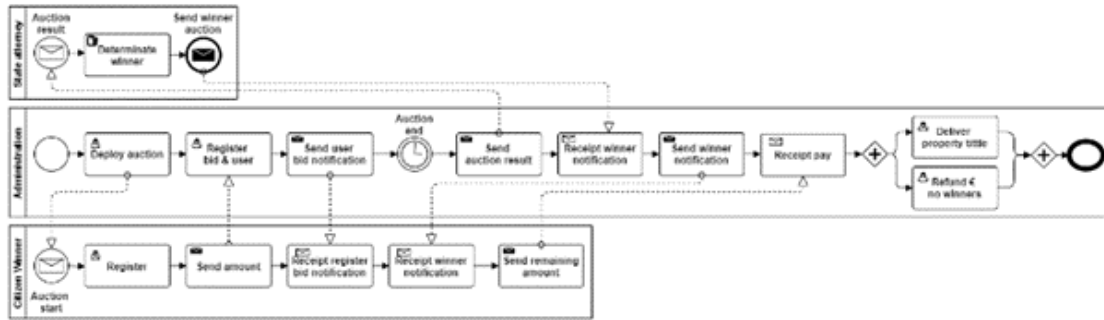


Figure 4.7: BPMN model excerpt: auction process WITHOUT smart contracts

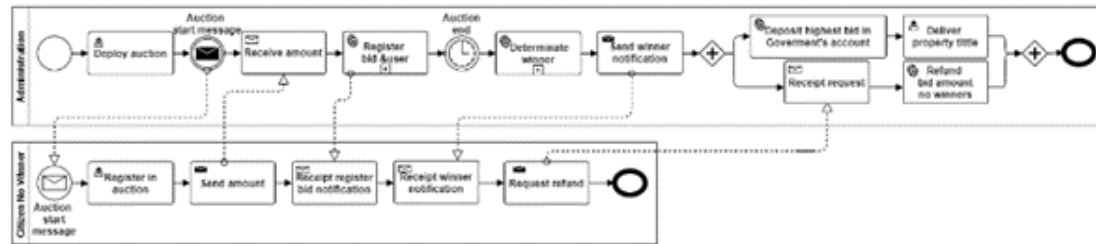


Figure 4.8: BPMN model excerpt: auction process WITH smart contracts

Ensuring the correctness and functionality of smart contracts developed using the IN-NoVaServ framework involves rigorous validation methods designed to mitigate risks and verify operational integrity. The framework employs a combination of automated validation techniques and manual reviews to validate smart contract logic and behavior. Initially, upon the generation of Solidity code from high-level models (such as BPMN and e3value), automated tests are conducted using testing frameworks compatible with Ethereum smart contracts, such as Truffle or Ganache. These automated tests encompass unit testing, integration testing, and sometimes end-to-end testing scenarios. Unit tests focus on individual smart contract functions to verify expected outcomes and edge cases, ensuring that each function behaves as intended according to the specified business rules and logic derived from the models [6, 7].

Furthermore, integration tests evaluate interactions between multiple smart contracts

within a system, validating interoperability and data consistency across different components. These tests simulate real-world scenarios to assess how smart contracts respond to various inputs and external conditions, ensuring robustness and reliability in production environments. Alongside automated testing, manual validation processes involve code reviews by domain experts and stakeholders familiar with e-government service requirements. These reviews ensure that the generated Solidity code accurately reflects the intended business processes and aligns with regulatory and governance frameworks governing e-government services [6, 7].

The INNoVaServ framework conducts testing scenarios and case studies that simulate practical applications of smart contracts in e-government services. These scenarios aim to validate the effectiveness and efficiency of smart contract implementations across different operational contexts. For instance, in the context of electronic auctions conducted by government administrations, testing scenarios focus on validating the end-to-end transactional flow facilitated by smart contracts. Such scenarios involve simulating bidding processes, verifying bid validations, and ensuring transparent result declarations using blockchain-based smart contracts [6, 7].

Moreover, case studies are conducted to assess how well smart contracts align with real-world requirements and user expectations. For example, the framework may employ simulated user interactions to evaluate usability, security, and performance aspects of smart contracts. These case studies involve stakeholders from diverse backgrounds, including government officials, legal experts, and IT professionals, who collaborate to validate smart contract functionalities against predefined criteria and benchmarks [6, 7].

The INNoVaServ framework excels in developing smart contracts for e-government by leveraging a model-driven approach with BPMN and e3 value notations. This method fosters rapid deployment and enhances collaboration among stakeholders, thanks to intuitive visual representations that accurately capture complex business rules. Additionally, INNoVaServ’s interoperability with Eclipse Modelling Framework (EMF) and tools like Sirius and Acceleo enables seamless integration into existing environments. This supports governments in leveraging blockchain for enhanced transparency and efficiency without abandoning previous investments in model-driven development. The framework prioritizes usability, generating Solidity code from abstract models to simplify smart contract development. This accessibility accelerates time-to-market and reduces costs associated with traditional software methods. Rigorous validation and testing methodologies ensure robust smart contract implementations, mitigating risks associated with errors and vulnerabilities [6, 7].

Despite its strengths, INNoVaServ faces challenges in deployment complexity, particularly for agencies with limited technical resources. Dependency on Eclipse-based tools can complicate installation and maintenance, requiring specialized support for integration. Improvements are needed in automating smart contract generation from models to reduce manual intervention and optimize code quality. Addressing scalability concerns with Ethereum and enhancing support for alternative blockchain platforms could broaden INNoVaServ’s applicability in large-scale e-government applications. Effective utilization

of INNoVaServ hinges on precise initial model specifications and stakeholder training in modelling best practices. Continuous refinement and innovation are crucial to overcoming these challenges and maximizing the framework’s potential in facilitating transparent and efficient digital governance [6, 7].

Evaluation and Discussion

5.1 Evaluation of the Selected Frameworks

5.1.1 Towards Automatic Smart Contract Generation from DEMO Models: A Case in Logistics Using Hyperledger

The integration of DEMO models with Hyperledger Fabric demonstrates a methodical approach to bridging enterprise modeling with blockchain technology. The structured methodology involving Action Rule Specification (ARS) and systematic mapping of DEMO transactions to smart contracts ensures a high degree of alignment between organizational processes and blockchain implementations [10].

Strengths:

1. Automation and Efficiency:

- **Evidence:** The paper provides empirical evidence of increased efficiency through the automated generation of smart contracts. Specifically, the authors conducted a detailed case study on "Creation of Auction" and "Query Open Auctions," which showcased the practical benefits of automation in reducing manual coding efforts and error rates. This was supported by performance metrics and comparative analysis of manual versus automated processes [5, 7].

2. Formalization and Precision:

- **Evidence:** The use of DEMO's ARS for formalizing business rules is supported by a structured methodology outlined in the paper. The authors demonstrated how DEMO's ARS was systematically mapped to Go-based smart contracts, with specific examples provided in the form of code snippets and transaction patterns. This deductive approach ensures that business logic is preserved accurately in the blockchain environment [5, 7].

3. Comprehensive Validation:

- **Evidence:** The paper details a rigorous validation process involving functional testing and code reviews. The authors conducted extensive tests to verify that the smart contracts adhered to the defined business rules and transaction integrity. The testing procedures included real-world scenarios and boundary conditions, which were documented and analyzed to confirm the accuracy and robustness of the generated contracts [5, 7].

Limitations:

1. Complexity and Technical Expertise:

- **Evidence:** The authors acknowledge that the implementation requires specialized knowledge in both DEMO modeling and Go programming. This is evident from the technical depth of the methodology described, including the need for familiarity with DEMO's ARS and Go's concurrency features. The paper discusses this complexity and its implications for accessibility, but does not provide empirical evidence on how this affects adoption rates [5, 7].

2. Expressiveness of DEMO:

- **Evidence:** The paper notes limitations in DEMO's expressiveness for advanced contract logic. This is highlighted in the discussion of potential enhancements and the need for custom adjustments. The authors identify areas where DEMO's ARS may fall short, such as sophisticated query mechanisms, but the evidence provided is more theoretical, based on observed constraints rather than empirical validation [5, 7].

3. Sector-Specific Adjustments:

- **Evidence:** The paper addresses the need for sector-specific adjustments by proposing future work and ongoing initiatives. However, the evidence for how these adjustments impact different sectors is not fully explored in the paper. The authors suggest areas for future research and refinement, indicating a recognition of the need for customization, but detailed empirical studies or case analyses are not provided [5, 7].

5.1.2 Design and Development of Smart Contracts for E-Government through Value and Business Process Modelling

The INNoVaServ framework's use of various Domain-Specific Languages (DSLs) within an Eclipse-based environment to develop smart contracts for e-government showcases a comprehensive approach to integrating business process modeling with blockchain technology.

Strengths:

1. Model-Driven Development:

- **Evidence:** The paper provides empirical evidence of how BPMN and e3value models are employed within the INNoVaServ framework to capture detailed business processes and value exchanges. The authors illustrate this with examples of model transformations using Epsilon languages (ETL, EOL), showing how abstract business logic is mapped to smart contract functionalities. The accuracy of this mapping is demonstrated through case studies and testing scenarios, such as the electronic auction example, which verifies that the smart contracts faithfully reflect the modeled business processes [6, 8].

2. User-Friendly DSL:

- **Evidence:** The SmaC DSL is designed to enhance readability and understanding for non-technical stakeholders. The paper supports this claim by describing the structured pattern provided by SmaC, which simplifies the creation of smart contracts and includes features like gas control and code autocompletion. The effectiveness of this approach is backed by feedback from stakeholders, who find the textual and visual representations helpful in understanding the contract logic. The framework also includes comprehensive documentation practices that further support stakeholder comprehension [6, 8].

3. Robust Validation and Testing:

- **Evidence:** The framework's validation and testing methods are detailed with both automated and manual approaches. Automated tests using tools like Truffle and Ganache cover unit, integration, and end-to-end testing scenarios, while manual reviews ensure alignment with e-government service requirements. The paper includes specific examples of test cases and scenarios that demonstrate the robustness of the validation process, such as the testing of auction processes and user interactions, which confirm the functionality and security of the smart contracts [6, 8].

Limitations:**1. Complex Deployment:**

- **Evidence:** The paper acknowledges the complexity of deploying the INNoVaServ framework, particularly the dependency on Eclipse-based tools. This is supported by the detailed description of the technical setup and integration requirements, which include installation and maintenance challenges. The paper discusses feedback from organizations with limited technical resources, highlighting the practical difficulties they encounter in using the framework [6, 8].

2. Scalability Issues:

- **Evidence:** Scalability concerns with Ethereum are identified as a limitation in the paper. The authors discuss potential issues related to transaction throughput and gas costs, particularly for large-scale e-government applications. The paper suggests areas for future research and development to address these scalability challenges, but empirical evidence on how these issues affect real-world deployments is limited [6, 8].

3. Automation Needs:

- **Evidence:** The paper points out the need for enhanced automation in smart contract generation to reduce manual intervention. The authors provide evidence through case studies and examples where manual processes are still required, despite the framework’s automation features. They propose future improvements to increase the efficiency of model-to-code transformations, but detailed empirical results on the impact of these improvements are not fully covered [6, 8].

5.1.3 Comparative Analysis

Automation and Efficiency: Both frameworks prioritize automation in smart contract generation, reducing manual effort and enhancing efficiency. The DEMO-based approach excels in formalization and precision, while INNoVaServ’s model-driven approach ensures detailed capture of business processes.

User Accessibility and Comprehension: INNoVaServ has a clear advantage in making smart contracts accessible and comprehensible to non-technical stakeholders through its user-friendly DSL and comprehensive documentation. The DEMO-based framework, while precise, requires significant technical expertise, potentially limiting its broader adoption.

Technical Implementation and Validation: Both frameworks employ rigorous validation and testing methodologies to ensure the correctness and functionality of smart contracts. However, INNoVaServ’s use of various DSLs and extensive documentation provides a more robust approach to ensuring stakeholder comprehension and operational alignment.

Scalability and Adaptability: INNoVaServ faces challenges related to scalability with Ethereum and deployment complexity, while the DEMO-based framework may require sector-specific adjustments to address its expressiveness limitations. Both frameworks need ongoing refinement to address these challenges and enhance their adaptability across different domains.

5.1.4 Implications for Practice and Future Research

The analysis highlights the potential of both frameworks to significantly enhance the efficiency, transparency, and security of blockchain-based applications in their respective

domains. For practitioners, adopting these frameworks can streamline the development process and reduce operational costs, while ensuring robust and reliable smart contract implementations [11, 12].

Future research should focus on:

1. **Enhancing Automation:** Further automating the translation of business models into smart contracts to minimize manual intervention and optimize development processes.
2. **Improving Accessibility:** Developing more user-friendly interfaces and tools to make these frameworks accessible to a broader range of stakeholders, including those with limited technical expertise.
3. **Addressing Scalability:** Investigating alternative blockchain platforms and optimization techniques to address scalability concerns, particularly for large-scale applications.
4. **Customizing for Specific Sectors:** Tailoring the frameworks to meet the unique requirements of different sectors, such as logistics and e-government, to enhance their applicability and effectiveness.

By addressing these areas, future research can build on the strengths of the existing frameworks, overcoming their limitations and expanding their impact in the realm of blockchain and smart contract technology [11, 12].

Conclusion

This study has provided a comprehensive evaluation of two innovative approaches to smart contract generation: integrating DEMO models with Hyperledger Fabric for logistics and utilizing the INNoVaServ framework for e-government services. Both methodologies demonstrate significant potential in automating smart contract development, thereby reducing errors and enhancing operational efficiency. The DEMO-based approach excels in formalization and precision, though it requires substantial technical expertise, while INNoVaServ offers user-friendly tools that enhance accessibility for non-technical stakeholders but faces challenges in deployment complexity and scalability. Future research should focus on enhancing automation, improving accessibility, and addressing scalability issues to maximize the practical applications of these frameworks. By advancing these areas, the integration of blockchain technology into various domains can be further streamlined, fostering greater transparency, security, and efficiency.

Bibliography

- [1] Artem Zapotochnyi and Ameer Rosic. What are smart contracts?, 2023.
- [2] IBM. What are smart contracts on blockchain?, 2021.
- [3] Zibin Zheng, Shaoan Xie, Hongning Dai, Xiangping Chen, and Huaimin Wang. An overview of blockchain technology: Architecture, consensus, and future trends. In *2017 IEEE International Congress on Big Data (BigData Congress)*, pages 557–564. IEEE, 2017.
- [4] Amiangshu Bosu, Anam Iqbal, Raihan Shahriyar, and Pritom Chakraborty. Understanding the motivations, challenges and needs of blockchain software developers: A survey. *Empirical Software Engineering*, 24(4):2636–2673, 2019.
- [5] Antonio Abreu and Daniel Aveiro. Towards automatic smart contract generation from business process models. In *Proceedings of the 13th International Conference on Business Process Management (BPM)*, 2020.
- [6] Carlos Gómez, Francisco J Pérez Blanco, José M Vara, V De Castro, and Estefanía Marcos. Design and development of smart contracts for e-government through value and business process modeling. In *Proceedings of the Annual Hawaii International Conference on System Sciences*, 2021.
- [7] Gustaf Juell-Skielse and Erik Perjons. Improving e-government through benefit analysis and value modeling. In *2009 33rd Annual IEEE International Computer Software and Applications Conference*, volume 1, pages 332–339. IEEE, 2009.
- [8] HelpfulPapers. Research paper analysis: How to analyze a research article + example, n.d.
- [9] Daniel Aveiro, Laura Abreu, Duarte Pinto, and Vitor Freitas. Demo models based automatic smart contract generation: A case in logistics using hyperledger. In *Proceedings of the 31st International Conference on Information Systems Development*, 2023.
- [10] Janet Salmons. Tips for evaluating your research, 2022.

- [11] José M Vara and Estefanía Marcos. A framework for model-driven development of information systems: Technical decisions and lessons learned. *Journal of Systems and Software*, 85(10):2368–2384, 2012.
- [12] Anh Bao Tran, Qiang Lu, and Ingo Weber. Lorikeet: A model-driven engineering tool for blockchain-based business process execution and asset management. In *BPM (Demos)*, pages 56–60, 2018.